

DeployGuard

Panel Question Bank - 140 Questions

Final Year Project II - Implementation

Project	DeployGuard - Deterministic PaaS Deployment Control Plane
Group Members	Hasan Sajjad Faria Fatima Tania Khawar
Department	Department of Computer Science, Air University Islamabad
Implementation Baseline	GitHub branch test2 - reviewed 13 September 2026

Implementation-focused viva preparation.

How to use this document

Practice answering each question in 20-60 seconds. Start with the architectural answer, then give one implementation example. Keep the project contribution separate from standard tools and cloud services.

Project, Problem and Contribution

1. What is DeployGuard in one sentence?

2. What real problem does DeployGuard solve?

3. What is the main contribution of your FYP?

4. Why is this more than a CI/CD pipeline?

5. Why did you choose a PaaS problem?

6. Who is the target user?

7. What is the FYP-II focus compared with FYP-I?

8. What is currently in scope?

9. What is out of scope?

10. What makes the system academically defensible?

Architecture and Control Plane

11. Explain the architecture at a high level.

12. What is the control plane?

13. What is the data plane/runtime?

14. Why separate control plane from runtime?

15. What is the source of truth for infrastructure?

16. What is the source of truth for the active release?

17. Why have multiple authorities at all?

18. How do you prevent Terraform and ECS from fighting?

19. What happens when the platform restarts?

20. Why is state management important in PaaS?

21. What does deterministic mean in your project?

22. What is a canonical contract?

23. Why not recalculate everything at each stage?

24. How is immutability used?

25. What is the benefit of fingerprints?

26. How is multi-service architecture handled?

27. How do you avoid one service contaminating another?

28. Why fail closed on ambiguity?

29. What is the role of queues?

30. What database stores control-plane state?

Repository Analysis and Build

31. How does DeployGuard know what to build?

32. What fields are important in the build target?

33. Which deployment contracts are currently supported?

34. How are JavaScript monorepos handled?

35. Which JavaScript package managers are recognized?

36. How are Python services handled?

37. How does the build subsystem work?

38. Why keep the build engine behind an abstraction?

39. Does DeployGuard require a user Dockerfile?

40. Do you support a fallback build path?

41. How do you resolve the service port?

42. What happens when two different ports are detected at the same priority?

43. What happens when no port can be resolved?

44. Which common framework defaults are known?

45. Why does the platform own PORT and HOST?

46. What prevents repository paths from escaping the repo?

47. Why pin the exact Git commit SHA?

48. What is an immutable image?

49. Why validate an image before deployment?

50. Can DeployGuard deploy a pure HTML/CSS site?

GitHub and CI/CD

51. Why use GitHub Actions?

52. How does DeployGuard trigger deployment?

53. Why use OIDC instead of AWS keys in GitHub?

54. What does the AWS trust policy verify?

55. What if the trust policy is wrong?

56. How do you know the workflow has enough AWS permissions?

57. Which AWS areas are covered by the capability model?

58. Why make permissions lifecycle-specific?

59. What if the workflow never starts?

60. How is workflow evidence correlated back to DeployGuard?

61. What happens if a GitHub step fails?

62. Do you depend on branch names after dispatch?

63. How are concurrent operations controlled?

64. Why is external action pinning relevant?

65. What is the difference between workflow execution and release truth?

AWS and Infrastructure as Code

66. Why AWS ECS Fargate?

67. Why not Kubernetes for the FYP?

68. Why use Terraform?

69. What resources does Terraform create?

70. Where is Terraform state stored?

71. Why is remote state important?

72. What is configuration drift?

73. How does DeployGuard reduce drift?

74. What is the ALB doing?

75. Why use security groups per service?

76. How does the runtime know if an app is reachable?

77. Why have a platform health path separate from the app path?

78. How are logs collected?

79. How are secrets delivered to ECS?

80. Why use an ECS execution role?

81. What is Cloud Map used for?

82. Why is EFS used?

83. How do you avoid exposing the database publicly?

84. What happens if Terraform plan fails?

85. What happens if Terraform apply fails?

86. What happens to partially created resources?

87. Why is destroy still Terraform-owned?

88. Can later Terraform runs revert an active application release?

89. What is infrastructure namespace/project scoping for?

90. How would you reduce AWS cost for thousands of apps?

Databases, State and Release Lifecycle

91. Which managed databases are supported?

92. How are database credentials created?

93. How is database persistence handled?

94. How does an application receive database configuration?

95. What if the application expects a driver-specific database URL scheme?

96. Why can only one service own the managed database attachment in the current contract?

97. What special case exists for MySQL?

98. What is a deployment generation?

99. Why have generations?

100. What states can a generation have?

101. Can two generations be live at once?

102. What is a stable release?

103. When is a release promoted?

104. What is redeploy?

105. How is rollback implemented?

106. Does rollback rebuild the old source?

107. Why is no-rebuild rollback safer?

108. What if rollback verification fails?

109. What does Reset & Deploy Fresh mean?

110. How are stale locks or interrupted operations handled?

Security and DevSecOps

111. What are the main security layers?

112. What is Trivy used for?

113. Can the security scanner block a deployment?

114. Why not scan only source code?

115. How do you prevent secrets appearing in AI prompts?

116. How do you prevent secrets in logs?

117. What is RBAC used for?

118. Why are audit logs important?

119. How do you secure destructive actions?

120. Why is least privilege difficult in a PaaS?

121. How does capability verification help least privilege?

122. What is the benefit of OIDC credential lifetime?

123. How do you handle unsupported or suspicious repository structures?

124. Does containerization make the app fully secure?

125. What future security improvements would you add?

Observability, AI, FinOps and Product

126. What observability does DeployGuard provide?

127. Why distinguish monitoring from troubleshooting?

128. How does AI troubleshooting start?

129. What evidence can the AI use?

130. Is AI allowed to decide the authoritative root cause?

131. What happens if the AI provider is down?

132. How are AI sessions audited?

133. What is FinOps in DeployGuard?

134. Why estimate cost before deployment?

135. What happens when a cost requires approval?

136. What billing features exist?

137. How is cost different from billing?

138. How would you measure platform success?

139. What is the biggest current scalability risk?

140. What is your strongest FYP-II demo sequence?